

Customer Churn ML Pipeline - Complete Guide

Customer Churn ML Pipeline

Complete Project Guide & Interview Handbook

Prepared for Sudama Pandit

SudamaPandit/ai-ml-customer-churn-data-platform

POSITIONING

The repository contains a valid local batch ML workflow with offline tests, SQL prediction history, monitoring calculations and an S3 adapter. It does not implement model persistence/registry, deployable orchestration, end-to-end Glue, automated retraining or production serving.

Clickable Contents

1. Executive summary

2. Beginner foundations

3. Architecture diagram

4. End-to-end data flow

5. Repository structure

6. Folder-wise code walkthrough

7. Quality, reliability and security

Clickable Contents (continued)

8. Testing and CI evidence

9. Gaps and critical risks

10. Production-ready redesign

11. Fifty interview questions

12. Rapid revision sheet

13. Glossary

Executive Summary

What this project is

A batch machine-learning example validates customer data, creates reproducible tenure-normalized features, trains a scaled class-weighted logistic regression model, scores churn probability and risk bands, calculates PSI/performance metrics, and can serialize predictions as Parquet to S3.

Business value

A retention team needs a prioritized list of customers at risk of leaving. The pipeline converts customer tenure, charges and support behavior into a reproducible risk score that can drive controlled outreach. Monitoring is necessary because feature distributions and model quality change over time.

60-second interview introduction

I built a batch churn-scoring implementation focused on reliable data inputs and consistent features. It validates schema and numeric ranges, uses the same feature code for training and inference, trains a scaled class-weighted logistic regression, returns ROC-AUC and average precision, assigns risk bands and calculates PSI and classification monitoring. Tests cover validation, reproducibility, scoring and monitoring. It is not a full MLOps deployment because registry, orchestration, temporal label construction and automated retraining are not implemented.

Honest status

The repository contains a valid local batch ML workflow with offline tests, SQL prediction history, monitoring calculations and an S3 adapter. It does not implement model persistence/registry, deployable orchestration, end-to-end Glue, automated retraining or production serving.

Beginner Foundations

Use this section to learn the vocabulary before studying the code. A data pipeline moves data through controlled stages. ETL means extract from a source, transform into a useful and reliable shape, and load into a destination.

- > Supervised learning trains from features plus a known label; churned is the target.
- > Training-serving skew occurs when training and scoring calculate features differently.
- > Class imbalance means churn examples may be rarer; `class_weight=balanced` adjusts model loss.
- > ROC-AUC measures ranking across thresholds; average precision is often informative for rare positive classes.
- > Drift means current feature distributions differ from the reference, but drift alone does not prove performance loss.

How to study the repository

First memorize the business flow, then trace one record through every component. Next learn which parts are actual runtime code, which are offline helpers, and which capabilities are missing. In interviews, evidence matters more than broad claims.

Architecture Diagram

The solid flow is implemented in code. Dashed or lighter notes mark external dependencies or incomplete production capabilities.

How to explain this diagram

- > The input schema contains customer_id, tenure, charges, tickets and the training label.
- > Validation rejects duplicates, missing/non-finite values, negative measures and non-binary targets.
- > One shared build_features function reduces training-serving skew.
- > Logistic regression outputs probabilities that are mapped to LOW/MEDIUM/HIGH bands.
- > Monitoring functions compute PSI and performance when labels are available; S3 publication is optional.

MONITORING LAYER: PSI, ROC-AUC, positive prediction rate, optional AI summary



End-to-End Data Flow

Step 1 - Validate training data

The quality function checks required columns, non-empty input, unique customer IDs, finite/non-negative numeric fields and a binary churn label.

Step 2 - Build features

tenure_months, monthly_charges and support_tickets are converted to numeric values. Charges and tickets are divided by tenure clipped to at least one month.

Step 3 - Split and train

The data is stratified into train/test sets. StandardScaler and LogisticRegression form one sklearn Pipeline with balanced class weights.

Step 4 - Evaluate

The held-out probabilities produce ROC-AUC and average precision. The function returns the trained Pipeline and metrics.

Step 5 - Score customers

The same feature builder is used, predict_proba creates churn_probability and pd.cut assigns LOW, MEDIUM or HIGH risk.

Step 6 - Monitor and publish

PSI compares reference/current numeric distributions; classification monitoring uses labels and probabilities. Predictions can be written as Parquet to S3 and represented historically in PostgreSQL.

Repository Structure

This table explains what every important folder contributes and whether it is used by the main runtime path.

Path	Purpose	Runtime truth
.github/workflows/ci.yml	Compile and pytest	Offline CI
src/data_quality.py	Training data contract	Called by train_model; tested
src/features.py	Shared features	Called by training and scoring
src/train.py	Split, scale, train, evaluate	Local implementation; tested
src/predict.py	Probability and risk bands	Local batch scoring; tested
src/monitor.py	PSI and classification metrics	Tested
src/storage.py	Parquet to S3 adapter	Implemented; not tested in CI
src/ai_insights.py	Optional Bedrock summary	Disabled by default
glue/prepare_features.py	Spark feature expressions	Not complete Glue entry point
sql/schema.sql	Prediction history table	External application
sql/analytics.sql	Risk-band reports	Not database tested
tests/	Quality, ML, monitoring, AI	Runs in CI

Folder-wise Code Walkthrough

src/data_quality.py

Applies a strict contract before training and returns basic dataset metrics.

```
numeric = df[NUMERIC_COLUMNS].apply(pd.to_numeric, errors="coerce")
if numeric.isna().any().any() or not np.isfinite(numeric.to_numpy()).all():
    raise ValueError("numeric features must be finite numbers")
```

Interview meaning: This prevents silent NaN/inf propagation and cryptic sklearn failures.

src/features.py

Validates base features and creates tenure-normalized ratios for both training and scoring.

```
denominator = result["tenure_months"].clip(lower=1)
result["charges_per_tenure_month"] = result["monthly_charges"] / denominator
result["tickets_per_tenure_month"] = result["support_tickets"] / denominator
```

Interview meaning: Shared code reduces training-serving skew, but feature semantics must still be point-in-time correct.

src/train.py

Validates class support, uses a stratified holdout and keeps preprocessing inside the model Pipeline.

```
Pipeline([
    ("preprocess", ColumnTransformer([("scale", StandardScaler(), FEATURE_COLUMNS)])),
    ("model", LogisticRegression(class_weight="balanced", random_state=seed))
])
```

Interview meaning: The fixed split is reproducible, but temporal/customer leakage and robust cross-validation still need consideration.

src/predict.py

Uses predict_proba and maps probabilities to operational bands.

```
probabilities = model.predict_proba(features)[: , 1]
pd.cut(probabilities,
    bins=[-0.01, 0.33, 0.66, 1.0],
    labels=["LOW", "MEDIUM", "HIGH"])
```

Interview meaning: Thresholds are illustrative and should be selected with business capacity/cost, then versioned.

src/monitor.py

PSI uses reference quantile bins with infinite outer edges so out-of-range current values are counted.

```
edges[0], edges[-1] = -np.inf, np.inf
return float(((cur - ref) * np.log(cur / ref)).sum())
```

Interview meaning: PSI is a drift signal, not direct evidence of model accuracy.

Continued

sql/schema.sql

Stores multiple prediction dates per customer using a composite primary key.

```
PRIMARY KEY (customer_id, prediction_date)
CHECK (churn_probability BETWEEN 0 AND 1)
CHECK (risk_band IN ('LOW', 'MEDIUM', 'HIGH'))
```

Interview meaning: The earlier single-customer key would overwrite history; the composite key preserves snapshots.

Quality, Reliability and Security

Input contract

Training rejects empty, duplicate, non-finite, negative and non-binary data. Scoring feature construction also validates feature inputs.

Feature consistency

Training and inference share `build_features`, reducing code skew. The separate Glue expressions still need parity tests.

Evaluation

A stratified holdout returns ranking metrics. There is no temporal split, cross-validation or calibration analysis.

Drift

PSI captures distribution movement including values beyond reference range. Thresholds and alert policy are not implemented.

Prediction history

SQL uses `customer_id` plus `prediction_date` as the primary key and constrains probability/risk values.

AI boundary

Bedrock receives aggregate monitoring metrics only and remains advisory; deterministic alerts should control operations.

Security baseline

No real credentials should be stored in Git. Runtime identities should use least-privilege roles, secrets should come from a managed secret backend, logs should avoid raw personal data, and CI permissions should remain read-only unless a workflow genuinely needs write access.

Testing and CI Evidence

A passing CI badge proves only the checks that CI actually executes. It is not proof of deployment or end-to-end service integration.

Area	Evidence	What it does not prove
Data quality	Missing, duplicate, negative, NaN, non-numeric and invalid labels	Real source extraction
Feature engineering	Expected columns and nonnegative ratios	Spark feature parity
Training	Reproducible run and bounded metrics	Generalization or production accuracy
Scoring	Probability range and risk-band vocabulary	S3/SQL publication
Monitoring	PSI identity/out-of-range and invalid probabilities	Alert integration
AI	Prompt and disabled configuration	Live Bedrock behavior

Best interview wording

I tested input validation, feature creation, deterministic training with a fixed seed, probability/risk-band output, PSI edge cases, classification monitoring and the safe disabled AI path. Those tests prove the local batch functions, not business effectiveness or deployment. I would add temporal validation, model serialization round trips, S3/SQL integration and staging orchestration before production.

Tests to add next

- > Temporal train/validation split using realistic observation and outcome dates.
- > Model serialization/load round-trip with schema and version metadata.
- > Feature parity test between pandas and Glue/Spark expressions.
- > S3 adapter test with botocore Stubber plus Parquet schema assertion.
- > PostgreSQL integration test for historical prediction upserts.

Gaps and Critical Risks

These are not reasons to hide the project. They are the strongest discussion points because they show that you can distinguish a portfolio implementation from a production system.

Severity	Risk	Consequence / response
Critical	No point-in-time label construction	Future information may leak into training; define observation and outcome windows.
High	No model persistence or registry	Scoring cannot reliably select/version an approved model; add artifact storage and promotion.
High	No deployable orchestration	Functions are not scheduled end to end; implement parameterized batch orchestration.
High	No production accuracy evidence	Synthetic fixtures only prove code behavior; validate on representative held-out data.
Medium	Illustrative fixed risk bands	Business actions may be inefficient; optimize/version thresholds by cost and capacity.
Medium	Glue/pandas feature drift	Separate expressions can diverge; add parity tests or one feature specification.
Medium	No alert workflow	Metrics are calculated but not acted on; define thresholds, owners and routing.
Medium	S3/SQL writes not integrated	Publication correctness is unproven; add contract and integration tests.

Production-Ready Redesign

Phase 1: Point-in-time dataset

Create observation dates, backward-looking features and future outcome labels; add leakage tests and dataset snapshots.

Phase 2: Model lifecycle

Serialize the full sklearn Pipeline, store schema/metrics/code version and add registry-style approval and rollback.

Phase 3: Orchestration

Build scheduled validation, feature, train/approve, score, publish and monitor tasks with idempotent batch IDs.

Phase 4: Evidence

Use temporal validation, calibration, subgroup/error analysis and model/data tests on representative data.

Phase 5: Operations

Publish versioned predictions, monitor drift/performance/freshness, alert owners and retrain only through controlled policy.

How to answer design follow-ups

Use Context -> Decision -> Trade-off -> Evidence -> Next step. State what the repository demonstrates, cite the exact test or implementation, acknowledge the boundary, and explain the production change. Never invent deployment history.

Fifty Interview Questions

The answers below are written as spoken interview responses. Learn the ideas rather than memorizing every word.

1 Explain the project end to end.

Interview-ready answer: The pipeline validates customer data, builds shared tenure-normalized features, trains a scaled class-weighted logistic regression, evaluates it, scores churn probability and risk bands, and computes drift/performance metrics. Predictions can be written to S3 and modeled historically in PostgreSQL.

2 Why logistic regression?

Interview-ready answer: It is fast, interpretable and provides a strong baseline for tabular churn data. I would compare it with tree-based models using temporal validation before choosing production performance.

3 Why StandardScaler?

Interview-ready answer: The numeric features use different units. Scaling stabilizes logistic-regression optimization and keeps coefficients more comparable.

4 Why keep preprocessing inside sklearn Pipeline?

Interview-ready answer: The fitted scaler and model travel together, reducing the risk that scoring uses different preprocessing from training.

5 How do you prevent training-serving skew?

Interview-ready answer: Training and score_customers both call build_features. I would also serialize the whole Pipeline and add parity tests against any Spark feature implementation.

6 Why divide by tenure?

Interview-ready answer: It normalizes charges and support activity by customer lifetime, distinguishing sustained rates from raw totals. Tenure is clipped at one to avoid division by zero.

7 Is clipping zero tenure always correct?

Interview-ready answer: It is a practical numerical rule, not a business truth. I would confirm new-customer semantics and potentially add an explicit new_customer flag.

8 How do you validate inputs?

Interview-ready answer: The contract checks required columns, non-empty data, unique IDs, finite/non-negative numeric values and a binary label before training.

9 Why class_weight=balanced?

Interview-ready answer: Churn is often imbalanced. Balanced class weights make errors on the minority class matter more, but I would still evaluate precision, recall and business cost.

Interview Questions (continued)

10 Why use stratify in train_test_split?

Interview-ready answer: It preserves class proportions in train and test and reduces the chance of an unusable holdout. It does not solve temporal leakage.

11 What metrics do you use?

Interview-ready answer: ROC-AUC measures ranking and average precision emphasizes positive-class retrieval under imbalance. For operations I would also choose precision/recall at the intervention capacity and calibration.

12 Is ROC-AUC enough?

Interview-ready answer: No. A model can rank well but produce poorly calibrated probabilities or weak precision at the selected threshold. Business decisions need threshold-specific and calibration metrics.

13 How are risk bands chosen?

Interview-ready answer: The current 0.33 and 0.66 thresholds are illustrative. Production thresholds should reflect retention capacity, false-positive cost and observed outcomes, and must be versioned.

14 What is PSI?

Interview-ready answer: Population Stability Index compares binned reference and current distributions. It is a drift indicator, not proof that model performance declined.

15 How does PSI handle out-of-range values?

Interview-ready answer: The implementation sets the outer bin edges to negative and positive infinity so current values outside the training range are still counted.

16 What if labels are delayed?

Interview-ready answer: I would monitor feature and prediction distributions immediately, then calculate performance metrics when the outcome window matures. Reports must use the correct label availability date.

17 Where is point-in-time correctness?

Interview-ready answer: The repository assumes an already prepared dataset; it does not construct temporal labels or snapshots. That is the biggest modeling risk and should be implemented before production claims.

18 How could leakage happen?

Interview-ready answer: Features might include activity after the prediction date or labels could overlap the observation window. I would define explicit observation/outcome windows and enforce timestamps in dataset SQL.

Interview Questions (continued)

19 Where is MLflow or a registry?

Interview-ready answer: It is not implemented. Training returns an in-memory Pipeline. I would add versioned artifact storage, metadata, approval and rollback rather than claim registry support.

20 How is the model deployed?

Interview-ready answer: It is not deployed. The repository implements local batch functions. Production needs model serialization, scheduled scoring, artifact selection and monitored publication.

21 How do predictions retain history?

Interview-ready answer: The SQL table uses customer_id and prediction_date as a composite primary key, allowing one snapshot per customer per date.

22 How would you prove reproducibility?

Interview-ready answer: Pin dependencies, version data/features/code, fix random seeds, serialize the full Pipeline and verify that the same inputs and artifact produce identical probabilities.

23 What happens if a feature is NaN or infinite?

Interview-ready answer: Validation rejects it with a clear error before sklearn runs. That gives a controlled data-quality failure instead of an opaque model error.

24 What is the Glue file?

Interview-ready answer: It contains equivalent Spark feature expressions for distributed preparation, but it is not a complete Glue entry point. I would add job arguments, input/output and parity tests before using it.

25 What is the first production change?

Interview-ready answer: Build a point-in-time correct training dataset and leakage tests. A well-deployed model trained on leaked data is still wrong.

26 Did you deploy this project?

Interview-ready answer: I describe it as a working portfolio implementation with passing unit CI, not as a production deployment. The repository proves specific code paths; a staging environment and integration evidence are still required.

27 What exactly did you test?

Interview-ready answer: I tested input contracts, feature calculations, deterministic local training, probability/risk output, PSI edge cases and classification monitoring. Those tests run offline and isolate deterministic behavior, so I do not claim they validate live external services.

Interview Questions (continued)

28 Why does green CI not prove the platform works?

Interview-ready answer: CI executes syntax and unit tests, but it does not automatically prove networking, IAM, service configuration, runtime compatibility or end-to-end data correctness. I would add integration and staging smoke tests for that evidence.

29 What is the biggest gap?

Interview-ready answer: The most important gap is absence of point-in-time label/feature construction and a model lifecycle. I would address it before calling the system production-ready.

30 How do you handle secrets?

Interview-ready answer: The repository should contain examples only. I would use workload identities or IAM roles for cloud access and a managed secret backend for database credentials, with least privilege and rotation.

31 How do you handle logging?

Interview-ready answer: I would use structured logs with run or batch identifiers, input and output counts, durations, quality results and error categories. I would avoid raw customer data and make logs searchable in a central platform.

32 How do you monitor data freshness?

Interview-ready answer: I would record the newest processed event or business timestamp and compare it with the freshness SLA. Breaches should alert with the affected dataset and last successful run.

33 How do you make retries safe?

Interview-ready answer: Every side effect needs an idempotency key or deterministic batch identifier. I would isolate run outputs, use upserts or transactional publication, and ensure a retry cannot duplicate or overwrite another run.

34 How do you support backfills?

Interview-ready answer: I would parameterize a bounded time range, use run-specific outputs, limit concurrency, reconcile every partition, and record completion in a control table so a failed slice can be resumed safely.

35 How do you prevent schema drift?

Interview-ready answer: I would define an explicit versioned contract, validate it at ingestion, quarantine incompatible records, test producer-consumer compatibility and allow only reviewed evolution.

36 How would you improve CI?

Interview-ready answer: I would keep fast unit checks, add linting and dependency scanning, then add targeted integration jobs using ephemeral infrastructure. Live staging smoke tests should run separately with tightly scoped credentials.

Interview Questions (continued)

37 How would you test external services?

Interview-ready answer: I would combine client stubs for request contracts, local or containerized substitutes where accurate, and a small real staging test for service-specific behavior that emulators cannot prove.

38 How do you protect customer data?

Interview-ready answer: I would minimize collected fields, encrypt in transit and at rest, restrict access by role and prefix/table, mask sensitive fields, set retention policies and audit access.

39 How do you estimate capacity?

Interview-ready answer: I would measure arrival rate, record size, daily volume, peak concurrency, transformation cost and downstream throughput. Those numbers drive partitioning, compute sizing and retention choices.

40 What happens when data quality fails?

Interview-ready answer: Production should stop publication for critical failures, persist rejected records with reason codes, alert the owner and support corrected replay. Warning-level rules can continue with explicit metrics.

41 How do you prove lineage?

Interview-ready answer: I would attach run ID, source window, input location, code version and output location to a run manifest and integrate those records with a catalog or lineage platform.

42 What would you document for operations?

Interview-ready answer: I would document SLAs, owners, dependencies, restart and backfill procedures, common failure signatures, data contracts, access requirements and rollback steps.

43 What would fail first at scale?

Interview-ready answer: The first bottleneck is likely local pandas feature preparation and the absence of distributed, versioned orchestration. I would validate that with load tests and redesign before increasing volume.

44 How do you control cost?

Interview-ready answer: I would right-size compute, prune partitions, compact small files, apply storage lifecycle rules, cap retries and timeouts, and publish per-run cost and volume metrics.

45 What would you change in your first week?

Interview-ready answer: I would first add evidence around the highest-risk path: define temporal observation/outcome windows, add leakage tests, serialize/version the Pipeline and test publication. Then I would update the runbook and architecture based on measured results.

Interview Questions (continued)

46 How do you explain a limitation without weakening your interview?

Interview-ready answer: I state the boundary directly, explain why the current choice was reasonable for a portfolio scope, cite what is actually tested, and give a concrete production solution.

47 Why should an interviewer value this project?

Interview-ready answer: It shows that I understand the complete data lifecycle and can review my own design critically. I can discuss implementation, tests, failure modes and realistic hardening rather than only drawing an architecture.

48 How would you perform a production readiness review?

Interview-ready answer: I would review correctness, recoverability, security, scalability, observability, deployment, cost and ownership. Each requirement needs evidence, not only code presence.

49 What is your rollback strategy?

Interview-ready answer: I would version code and data outputs, publish atomically, retain the previous valid version, and make consumers switch through a controlled pointer or view. Rollback steps must be tested.

50 Give the honest one-line status.

Interview-ready answer: This is a compact, technically useful portfolio implementation with verified unit behavior and clearly documented production gaps; it is not a fully deployed platform.

Rapid Revision Sheet

One-minute memory map

- > Flow: validate -> shared features -> sklearn Pipeline -> probabilities/risk bands -> monitor -> optional S3.
- > Model: scaled, class-weighted logistic regression.
- > Metrics: ROC-AUC and average precision; thresholds remain illustrative.
- > Monitoring: PSI plus label-based metrics when outcomes exist.
- > Testing truth: local functional behavior, not model business value or deployment.
- > Top risk: no point-in-time label construction; possible leakage if source data is prepared incorrectly.

Safe sentence when challenged

The repository demonstrates the design and unit-tests specific offline behavior. It does not prove every external integration or a production deployment. I can show exactly what is verified and what I would add to prove the remaining path.

Glossary

Term	Meaning
Churn	Customer stops or is likely to stop using a service.
Feature	Input variable used by a model.
Label	Known outcome used for supervised training.
Leakage	Training uses information unavailable at prediction time.
Pipeline	Bundled preprocessing and model steps.
ROC-AUC	Ranking quality across classification thresholds.
Average precision	Precision-recall summary useful for imbalance.
Calibration	Agreement between predicted probability and outcome frequency.
PSI	Population Stability Index for distribution drift.
Model registry	Versioned model artifacts and lifecycle metadata.